HYPERGRID

Design and Architecture

Practical Reference · Version 1.0 · March 2026 · Confidential — Internal Use Only

Document	Hypergrid Design and Architecture v1.0
System	Hypergrid N-Dimensional Distributed Spreadsheet System (NDSS)
Platforms	Apapo · Kogi (IW-OS) · Ume (B-OS) · Qala (SF-OS)
Version	1.0 · March 2026
Status	Authoritative Reference — Active Design
Classification	Confidential — Internal Use Only

Abstract

Hypergrid is the substrate — not an application. It provides the universal data layer on which Kogi, Ume, and Qala are all built. Its central claim is that every domain entity is a row in an N-dimensional distributed spreadsheet, and that building three distinct domain operating systems on one shared substrate is better than maintaining three separate storage stacks.

This document is the practical design and architecture reference for Hypergrid. It covers what the system is, how the four-layer architecture works, the data model, the core modules, CRDT semantics, the write path, and a complete honest assessment of the technical concerns. Crucially, it specifies a tiered deployment model — Embedded, Collaborative, Enterprise — so that the infrastructure grows with the workload rather than starting at full production cost. The document incorporates all five remedies for overhead: tiered storage backends, lazy service activation, in-process AI for simple signals, Hybrid Logical Clocks to replace the 255-node VectorClock ceiling, and the Lattice CRDT for lifecycle states shipped in v1 rather than deferred to v2.

> **Design principle:** Start with the minimum viable infrastructure. The Grid runtime, the cell store, and the CRDT engine are the core. Everything else — Kafka, ClickHouse, Meilisearch, the AI pipeline, the federation coordinator — activates only when the feature that requires it is first exercised.

1. What Hypergrid Is

A conventional spreadsheet generalizes well. Dimension 1 is the entity (rows). Dimension 2 is the field name (columns). That gets you a relational table. Now add: dimension 3 as a time period axis, or a jurisdiction axis, or an audience segment axis — without changing the schema. Add per-attribute CRDT semantics so concurrent writes from different nodes resolve without coordination. Add an append-only EventLog that serves as audit trail, AI feature store, and time-travel query substrate simultaneously. Add a typed property graph where every entity is automatically a graph node. Add a query language that works across all dimensions. The result is Hypergrid.

The system is a synthesis of five well-understood components whose combination is novel. N-dimensional OLAP models, event sourcing, CRDT-based distributed consistency, property graphs, and AI computed attributes each have decades of prior art. The coherent integration of all five into a single substrate — where one entity is simultaneously a spreadsheet row, an EventLog subject, a graph node, and an AI computation target — is the architectural contribution.

What is genuinely new

Innovation	Why it matters
Per-attribute CRDT semantics	Most CRDT systems operate at the row or document level. Hypergrid assigns independent merge semantics to each attribute: owners uses OR-Set, status uses Lattice, budget_spent uses PNCounter, health_score uses LWW with System-only write permission — all on the same entity row. Concurrent edits to different attributes never conflict.
EventLog serving three purposes simultaneously	Standard architectures need three separate systems for audit trail, time-travel, and AI feature store. Hypergrid collapses them: the EventLog is the primary record; the cell store is a materialized view of it.
OR-Set semantics on graph edges	Concurrent edge additions from different nodes both survive. Tagged removes only remove the specific add they reference. A distributed cooperative's members can independently add relationships without losing any.
Consent Lattice on CrossGridLink	None → Pending → Accepted|Declined; Accepted → Revoked; Revoked cannot return to Accepted without a new edge. The consent state is itself a CRDT — concurrent accept and revoke resolve deterministically.
Lattice CRDT for lifecycle governance	Invalid state transitions are a mathematical property of the data structure, not application code. A network partition cannot drive a governed entity into an illegal state.
ComputedFrom provenance edges	System-written by AI engines to record which exact cells were read to produce a computed attribute. No production lineage system today operates at individual-field granularity.

What is prior art, executed well

The N-dimensional model is OLAP cubes (30 years). AS_OF time-travel is Datomic and CockroachDB. TRAVERSE GRAPH is Cypher/Gremlin. DimFold/DimExpand is ROLLUP/PIVOT. The EventLog as source of truth is event sourcing since 2010. AI feature store writeback is standard MLOps (Feast, Tecton). The plugin architecture is Eclipse (2001). None of this is a criticism — executing prior art cleanly and integrating it coherently is engineering value.

2. The Four-Layer Architecture

Every domain system built on Hypergrid follows this four-layer structure. The substrate is shared across Kogi, Ume, and Qala. The bridge layer is the only place domain types ever touch Hypergrid primitives.

Layer	Component	Rule	Domain example
Presentation	APIs, UIs, SDKs, CLIs	Never exposes HyperCell or DimCoordinate to callers	GET /api/portfolios/:id — returns a PortfolioComponent, not a HyperRow
Domain logic	PortfolioSystem · UmeKernel · QalaOS	Knows nothing about cells or dimensions. Operates on typed domain structs.	portfolio_system.update_status(id, Status::Active, actor)
Bridge (codec)	ComponentStore · OrgModuleStore · SolutionStore	The only place domain types map to Hypergrid primitives. Implements bootstrap(), write(), read(), exists() only.	ComponentStore::write() serialises a Component struct into batch write_cell calls
Substrate	Grid · Hypercubes · Hypergraph · EventLog · CRDT engine · HyperQL · AI plugins	Shared. Zero domain knowledge. Operates entirely on UUIDs and typed attribute values.	grid.write_cell_batch(cube_id, ops, actor)

The isolation rule: Domain logic and domain types must never appear below the bridge layer. The substrate layer handles UUIDs and typed values, nothing else. This separation is what makes it possible to add a fourth domain system later without touching Hypergrid.

Storage layer (below substrate)

Service	Role	When required
PostgreSQL	Primary cell store + EventLog + graph edges + namespace registry	Always — this is the floor requirement for Collaborative and Enterprise tiers
Redis	Cell cache (L1) + CrdtLog hot path + WebSocket session state	Collaborative tier and above; not required for Embedded
Kafka	Federation delta broadcast + AI compute request routing	Only when a second node is added OR a remote AI engine is registered
ClickHouse	Analytics backend for DimFold/DimExpand at > 1 million cells	Activated on first query that exceeds the PostgreSQL threshold
Meilisearch	Full-text and faceted search across HyperRows	Activated on first searchable Hypercube registration
S3 / object store	EventLog cold archive + Hypercube snapshots	Activated when EventLog hot partition reaches 500K entries
SQLite	Embedded single-process cell store (Embedded tier only)	Solo/mobile/desktop/offline deployments; zero additional infra

3. The Data Model

Every entity in every domain system maps to the same universal layout. D_1 is the entity UUID. D_2 is the field name string. The HyperCell at coordinate (D_1, D_2) holds the typed value. Higher dimensions extend the schema for time-series, jurisdictional, segmented, or multi-version data — without changing the cell schema or the storage engine.

Dimension axes

Dimension	Axis type	Key type	Encodes	Example
D_1	EntityAxis	UUID	The entity — every domain root concept	kogi.portfolio.components: ComponentId
D_2	PropertyAxis	String	The field name — the 'column' of the spreadsheet	"name", "status", "budget_spent", "health_score"
D_3	TimeAxis	Timestamp	Time period — quarter, sprint, version	kogi.portfolio.kpis: "2026-Q2"
D_3	CategoryAxis	String	Jurisdiction, audience segment, risk tier	ume.chombo.entities: "UK", "EU-GDPR", "US-CA"
D_4	CategoryAxis	String	Second classification — metric category, environment	qala.solutions.metrics: "prod", "staging"

Axis discipline: Always start with N=2. Add D_3+ only when a cross-sectional query pattern is known and required — never speculatively. Never use a new axis for filtering that can be expressed as a D_2 attribute value filter. Never exceed N=6 for cubes with user-facing views.

The HyperCell

The HyperCell is the universal data atom. It lives at an N-dimensional coordinate and carries a typed attribute map, causal metadata, and a governance record. The TypedAttrValue enum covers every value kind the system needs to express.

```
// A HyperCell at coordinate (entity_id, "health_score")HyperCell {    coord:
(ComponentId, "health_score"),    attributes: {        "value":      AiSignal { value:
Number(82.4), model_id: "kogi.health.v2",                                  confidence: 0.87,
computed_at: ..., stale_at: ... },        "data_type":  Text("Number"),    },    vector_clock:
HlcTimestamp { wall_ms: 1711190400000, logical: 3, node: 1 },    version:        14,
last_actor:     "system:kogi-engine",    visibility:     Tenant,}// The same entity's name
cell is independent — different CRDT semantics,// different write permission, different actor
historyHyperCell {    coord:          (ComponentId, "name"),    attributes: { "value":
Text("Brand Identity — Client A") },    last_actor:     "user:alice@kogi.io",    ...}
```

Storage encoding

Three physical storage strategies exist. The right choice depends on axis density. Hybrid encoding is correct for nearly all primary entity cubes.

Encoding	Physical layout	Lookup	Use when
Dense	Full D_1×D_2 row per entity; JSONB column per field	O(1) primary key	Small, fully-populated N=2 cubes with bounded entity and field counts
Sparse	Key-value table; only non-null cells stored; BTree index	O(log n)	TimeAxis, GeoAxis, or any axis with > 100K keys — rows exist only for populated coordinates

Encoding	Physical layout	Lookup	Use when
Hybrid (recommended)	Dense base table for $D_1 \times D_2$ (the hot path); separate sparse extension table for D_3+	O(1) base, O(log n) extension	All primary entity cubes — kogi.portfolio.components, ume.kernel.modules, qala.solutions

The physical cell table

```
CREATE TABLE hypergrid_cells (    grid_id         UUID NOT NULL,    cube_id         UUID NOT
NULL,    dim1_key        UUID     NOT NULL,   -- EntityAxis (entity ID)    dim2_key
TEXT     NOT NULL,   -- PropertyAxis (field name)    dim3_key        TEXT,                --
TimeAxis / CategoryAxis (N≥3)    dim4_key        TEXT,                -- GeoAxis / TenantAxis
(N≥4)    attr_value      JSONB    NOT NULL DEFAULT '{}',    attr_crdt_ts    BIGINT   NOT NULL,
-- HLC timestamp (16 bytes)    last_actor      TEXT     NOT NULL,    last_mutated_at
TIMESTAMPTZ NOT NULL DEFAULT NOW(),    PRIMARY KEY (grid_id, cube_id, dim1_key, dim2_key,
dim3_key, dim4_key)) PARTITION BY HASH(grid_id);-- Hot read path: row cache by D₁CREATE INDEX
idx_cells_d1 ON hypergrid_cells (grid_id, cube_id, dim1_key);-- Full-text on attribute
valuesCREATE INDEX idx_cells_val ON hypergrid_cells USING GIN (attr_value);-- Time-range scan
for D₃ TimeAxisCREATE INDEX idx_cells_d3 ON hypergrid_cells USING BRIN (dim3_key)  WHERE
dim3_key IS NOT NULL;
```

4. Core Modules

Hypergrid is composed of 15 modules. Domain systems interact primarily with HG-CORE, HG-CRDT, HG-GRAPH, HG-QL, and HG-AI. The others are infrastructure or extension points. A module can be absent or in a degraded state without taking down the Grid — the core trio (HG-CORE, HG-CRDT, HG-QL) is the minimum viable substrate.

Module	Name	Responsibility	Required tier
HG-CORE	Core substrate	Grid runtime, Hypercube registry, DimensionAxis, HyperCell, HyperRow, EventLog, CrdtLog, HLC clock, PolicyEngine	Embedded+
HG-CRDT	Distributed consistency	Per-attribute CRDT merge engine. LWW, OR-Set, PNCounter, GrowOnly, MaxRegister, Lattice, AppendLog, DeepMergeJson	Embedded+
HG-GRAPH	Hypergraph	Typed directed edges, CrossGridLink, ShadowCell protocol, BFS/DFS traversal, DFS cycle detection	Embedded+
HG-QL	Query language	HyperQL — N-dimensional SELECT, DimSlice, FOLD, EXPAND, TRAVERSE GRAPH, AS_OF time-travel	Embedded+
HG-AI	AI intelligence	HypercubePlugin trait. Tier-1 formulas (synchronous). Tier-2 AI signals (async). In-process for simple; Kafka+gRPC for remote LLM	Embedded+ (in-process); Enterprise (remote)
HG-VIEW	View engine	HypercubeView — saved DimSlice projections, sort, group, pivot, DimFold, DimExpand, render adapters	Collaborative+
HG-SPACE	Spaces & workspaces	Bounded operational contexts with governance, member management, treasury. Personal, Team, Org, Federation types	Collaborative+
HG-ID	Identity & tenancy	Multi-tenant, multi-identity, multi-profile. TenantPartition, VisibilityMask, SplitPolicy, CrossTenantMerge	Collaborative+
HG-NS	Namespace system	Hierarchical URI addressing (kogi://worker/slug/entity/id/). Federation-aware resolution	Collaborative+
HG-COMP	Computation engine	Tier-1 synchronous formula evaluation. Rollup aggregation engine	Collaborative+
HG-DIM	Dimension system	Axis declaration, type registry, sparse/dense/hybrid encoding selection, cross-dim join planning	Collaborative+
HG-FED	Federation	CrdtLog delta sync over Kafka. CrossGridLink consent flow. ShadowCell sync routing. FederationCoordinator	Enterprise
HG-PLUGIN	Plugin system	HypercubePlugin trait for custom axis types, attribute types, AI engines, render adapters, data connectors	Collaborative+
HG-EXPORT	Export & integration	CSV, XLSX, JSON, Parquet, Arrow, GraphQL, webhook streaming of any N-dim view slice	Collaborative+

Module	Name	Responsibility	Required tier
HG-OPS	Telemetry & ops	OpenTelemetry tracing, Prometheus metrics, structured audit logs, health probes, federation dashboards	Collaborative+

5. CRDT Semantics

Every attribute key in every Hypercube must be assigned CRDT semantics at schema registration time. This is the most consequential design decision in the bridge layer. Wrong semantics cause silent data loss in federated deployments. The rule: assign semantics based on what the field means, not how it is implemented.

The decision tree: Is it a set? → OR-Set. Is it a lifecycle state? → Lattice. Is it an accumulator that goes both directions? → PNCounter. Is it always increasing? → GrowOnly. Is it a version number? → MaxRegister. Is it an ordered audit trail? → AppendLog. Is it a config blob needing partial-key updates? → DeepMergeJson. Otherwise → LWW.

CRDT type	Merge rule	Use when	Examples
LastWriteWins (LWW)	Higher HLC timestamp wins; NodeId lexicographic tiebreak	Single-valued scalar where the latest write is always correct	name, description, config blobs, reference IDs, boolean flags
OR-Set	All concurrent adds survive; removes are tagged — only remove the exact element they reference	Set-valued field where concurrent additions from multiple users/nodes must both survive	tags, owners, children, dependencies, members, approver_ids, toolbox_ids
GrowOnly Counter	Sum of all increments across all nodes; decrement rejected at write time	Monotonically increasing integers that can never meaningfully decrease	view_count, follower_count, restart_count, contribution_count
PNCounter	Sum of (positive ops − negative ops) per node, merged across nodes	Transactional accumulator that legitimately goes both up and down	budget_spent, hours_logged, allocated_units, stock_count
MaxRegister	Maximum value wins regardless of causal ordering	Version/sequence numbers that must never decrease	version, schema_version, sequence_number, sprint_number
Lattice	Merge to least upper bound in the declared partial order; no backward transitions without Admin override	Governed lifecycle states that must only advance — enforced at data layer, not application code	ComponentStatus, SolutionLifecycle, CCRStatus, ModuleState
AppendLog	All appends from all nodes survive; causally ordered by HLC	Ordered audit trails where every entry from every node must be preserved	activity_log, comment_thread, decision_history, governance_notes
LWW + System-only	LWW with PermissionTier::System — no human actor can write	AI-computed signals where humans must never overwrite the model output	health_score, risk_score, anomaly_flag, drift_status, compliance_prediction

The Lattice CRDT — ship in v1

The previous design deferred the Lattice CRDT to v2. This is a correctness gap, not a simplification. Under a network partition with LWW status, two nodes can independently drive the same entity to different lifecycle states — resolved by recency, not governance. For pharmaceutical approval, regulated financial instruments, or any CCR workflow, this is a regulatory risk, not a theoretical one.

	LWW status (v1 original)	Lattice status (this spec)
Partition scenario	Node A: solution = Approved; Node B (partitioned): solution = Draft	Node A: solution = Approved; Node B: solution = Draft

	LWW status (v1 original)	Lattice status (this spec)
Merge result	Whichever timestamp is higher wins — governance invariant violated by a network event	join(Approved, Draft) = Approved — state never regresses
Implementation cost	Zero	Define LatticeOrder struct with valid transitions; swap SetAttr for TransitionState in bridge write() for status fields
When to ship	— (deferred)	v1 — any domain system with governance-meaningful lifecycle states needs this on day one

Implementation: The Lattice merge engine is fully designed. The cost is: (a) a LatticeOrder struct defining the partial order, (b) a TransitionState variant in CrdtOperation, (c) calling TransitionState instead of SetAttr for status fields in the bridge layer. Two to three days of implementation, not a new system.

Causal clock: Hybrid Logical Clock, not VectorClock

The previous design used VectorClock = HashMap<NodeId, u64>. This has a 255-node hard ceiling and costs O(N) bytes per cell per node. Every HyperCell carries a VectorClock snapshot, so high-cardinality cubes with many federation nodes compound the storage cost. The correct replacement is a Hybrid Logical Clock (HLC).

Property	VectorClock (previous)	HLC (this spec)
Storage per cell	O(N) bytes — grows with node count	16 bytes always — wall_ms(8) + logical(2) + node_id(4) + padding(2)
Node count ceiling	255 hard limit	No ceiling — node_id is a 32-bit integer
Causality	Precise: a happened-before b iff A.counters ≤ B.counters on all nodes	Preserved: a happened-before b iff a.hlc < b.hlc or a.hlc == b.hlc and a.node < b.node
Federation delta sync	Per-node comparison across full clock	Time-range scan on wall_ms — simpler to implement and index in PostgreSQL
Prior art	Lamport clocks generalized to N nodes	CockroachDB, MongoDB, TiKV — production-proven

```
// HLC timestamp — 16 bytes totalpub struct HlcTimestamp {    pub wall_ms:  u64,   // wall-clock milliseconds since Unix epoch    pub logical:  u16,   // tie-breaker counter when wall_ms ties    pub node_id:  u32,   // 32-bit node identifier — no 255 ceiling    pub _pad:  u16,   // alignment padding}impl HlcTimestamp {    // Advance: take max(self.wall_ms, now_ms); if tied, increment logical    pub fn tick(self, now_ms: u64) -> HlcTimestamp { ... }    // happened_before: a < b iff a.wall_ms < b.wall_ms,    //                  or equal wall_ms and a.logical < b.logical    pub fn happened_before(self, other: HlcTimestamp) -> bool { ... }}// LWW merge rule using HLC — same semantics as VectorClock, better storagefn lww_merge(a: HyperCell, b: HyperCell) -> HyperCell {    if a.hlc > b.hlc { a }    else if b.hlc > a.hlc { b }    else { if a.node_id > b.node_id { a } else { b } }  // deterministic tiebreak}
```

6. The Write Path

Every write to any entity in any domain system flows through this exact eight-step sequence. Understanding it is essential for building correct domain systems and diagnosing latency issues.

Step	Name	Action	Blocking?
01	Permission check	actor.tier ≥ attr.write_permission required. Rejected writes produce a PermissionDenied EventLog entry. AI-computed attrs require PermissionTier::System.	Yes
02	Policy engine evaluation	GovernanceEngine evaluates attached PolicyEngine plugins. Results: Allow (continue) / Deny (return error) / RequireApproval (queue ApprovalRequest, return Pending).	Yes
03	CRDT operation construction	HlcTimestamp.tick(now_ms) called first. CrdtOperation constructed — SetAttr for LWW, AddToSet for OR-Set, IncrCounter for PNCounter, TransitionState for Lattice. Clock is stamped before the write.	Yes
04	Local merge (in-memory)	Merge engine applies CRDT semantics for this attr key. LWW losers produce a ConflictRecord (not an error). OR-Set adds are tagged with a new UUID. Lattice transitions are validated against the partial order — invalid transitions return Err(InvalidStateTransition).	Yes
05	Persistence (PostgreSQL)	UPSERT on (grid_id, cube_id, dim_keys...). For batch writes — always preferred — all N fields execute in one transaction. Batch is ~3× faster than N individual UPSERTs for a 30-field entity write.	Yes
06	EventLog append	Immutable entry: before_value, after_value, actor, hlc_timestamp, cube_id, dim_keys. This record is the audit trail and the AS_OF time-travel source. PostgreSQL RLS and application code both prevent UPDATE/DELETE.	Yes
07	CrdtLog append	Hot path: Redis LPUSH for low-latency federation pick-up. Background flush to PostgreSQL. Used by peer nodes for delta sync. Only active when a federation peer exists.	Yes (Redis)
08	Async fan-out	AI invalidation (in-process plugin queue or Kafka for remote engines). Cache invalidation (Redis key eviction). Search index update (Meilisearch). Federation broadcast (Kafka — only if peers exist). WebSocket push to collaborative editors. None of these block the response.	No

Batch write is the single most impactful optimization: grid.write_cell_batch(cube_id, ops, actor) executes all N field writes in one PostgreSQL transaction, one EventLog batch INSERT, one Redis LPUSH, and one AI invalidation notification. For a 30-field entity, batch is roughly 3× faster than individual write_cell calls. All bridge-layer write() implementations must use batch by default.

The read path (summary)

Step	Action	p50 latency
1	Redis L1 cache check — key = cell:{grid_id}:{cube_id}:{dim_hash}	< 1ms on hit
2	Computed attribute check — Tier-1 formulas evaluate synchronously; Tier-2 AI returns cached AiSignal or None if not yet computed	< 1ms (Tier-1); 0ms (Tier-2 cached)

Step	Action	p50 latency
3	PostgreSQL primary key lookup — O(1) on (grid_id, cube_id, d1_key, d2_key)	< 5ms on miss
4	VisibilityMask applied — caller's tier and identity filter which attributes are returned	< 1ms
5	Redis cache written — TTL: 30s hot attrs, 300s cold attrs	< 1ms

7. The Hypergraph

The Hypergraph is the relational layer of Hypergrid. Every HyperRow in every Hypercube is automatically registered as a HypergraphNode when the entity is created. Connections between entities — within a Grid or across Grids — are HypergraphEdges. Edges are first-class typed objects with their own attributes, consent state, and causal version.

Edge types

EdgeType	Consent	Cycle detection	Shadow cell	Minimum creator tier	Use
Hierarchy	No	No (diamonds valid)	No	Contributor	Parent-child structural containment — portfolio tree, org chart, product taxonomy
Dependency	No	DFS enforced — cyclic adds rejected	No	Contributor	Blocking: source cannot complete before target. Any cycle triggers HG-422C.
Association	No	No	No	Contributor	Soft lateral cross-reference — no blocking or ordering semantics
Contains	No	No	No	Editor	Ownership containment — child lifecycle subordinate to parent
CrossGridLink	Yes — required	No	Yes — on acceptance	Editor	Inter-Grid connection. The KLNK economic graph atom. Six-phase lifecycle.
Collaborates	Mutual (both parties)	No	Yes — bidirectional	Editor	Peer collaboration across Grids — both parties consent; shadow cells on both sides
Employs	Worker consent required	No	Yes	Manager	Directed: org → worker. Worker controls which attrs are mirrored.
InvestedIn	Target consent required	No	Yes	Editor	Economic stake — target controls what investor sees
ComputedFrom	System-written	No	No	System	Provenance: records exactly which cells an AI engine read to produce a computed attr
Custom(String)	Plugin-defined	Plugin-defined	Plugin-defined	Plugin-defined	Domain-specific relationship via HypercubePlugin EdgeTypePlugin interface

The CrossGridLink and ShadowCell protocol

CrossGridLink is the mechanism for all cross-system connections. The six phases are: (1) link request with proposed mirrored attrs; (2) target owner configures consent — narrows or approves the attr set; (3) ShadowCell provisioned in requesting Grid with initial snapshot; (4) real-time sync via Kafka shadow.sync topic as source entity changes; (5) optional write-back from requester to source for declared writeback_attrs; (6) lifecycle management — suspend, renegotiate, revoke, expiry.

Data sovereignty: The target entity owner always controls what is visible. The requesting Grid never has access to attributes not explicitly consented. Write-back requires explicit declaration in writeback_attrs. Revoking the link archives the ShadowCell but preserves the historical sync record in the EventLog.

Graph storage and traversal

Backend	Scale threshold	Traversal	Notes
PostgreSQL (v1 default)	< 1 million nodes	Recursive CTE (BFS, DFS, Dijkstra at app layer)	Zero additional infrastructure; shares the PG connection pool; adequate for all domain deployments below platform scale
Neo4j Enterprise (v2.5, KLNK only)	≥ 1 million nodes	Native Cypher + GDS (Louvain, PageRank, Dijkstra)	Activated only for KLNK economic graph at platform scale — not required for individual Grids

8. HyperQL

HyperQL is the query language for N-dimensional Hypercubes. It extends SQL with dimension-aware cell references, DimSlice filtering, dimension folding (DimFold) and expansion (DimExpand), graph traversal, and AS_OF time-travel. Every HyperQL query is executed by the HG-QL module against the Universal Cell Store and Hypergraph simultaneously.

Core query patterns

```
-- 1. Basic entity query — N=2 (most common)SELECT cell[D₁, 'name'].value, cell[D₁,
'status'].value,       cell[D₁, 'health_score'].valueFROM kogi.portfolio.componentsWHERE
cell[D₁, 'status'].value = '"Active"'  AND cell[D₁, 'item_type'].value = '"Project"'ORDER BY
cell[D₁, 'health_score'].value DESC;-- 2. AS_OF time-travel — replay EventLog to any prior
stateSELECT cell[D₁, 'status'].value, cell[D₁, 'budget_spent'].valueFROM
kogi.portfolio.components AS_OF '2026-03-01T00:00:00Z'WHERE cell[D₁, 'item_type'].value =
'"Project"';-- 3. DimFold — collapse time axis into aggregate (N=3 → N=2 result)SELECT
D₁.entity_id, FOLD D₃ WITH AVG AS avg_healthFROM kogi.portfolio.kpisWHERE D₂.metric_name =
'health_score'  AND D₃.period IN ('2025-Q3','2025-Q4','2026-Q1')ORDER BY avg_health DESC;-- 4.
TRAVERSE GRAPH — follow typed edgesTRAVERSE GRAPH FROM '<component_uuid>'  EDGE_TYPE =
Dependency DIRECTION = Inbound MAX_DEPTH = 10  FILTER WHERE cell[D₁, 'status'].value !=
'"Archived"';-- 5. Cross-system join via Employs edge (Ume → Kogi)SELECT ume_unit.org_unit_id,
cell[ume_unit.org_unit_id, 'name'].value AS unit_name,       COUNT(worker.worker_id) AS
contractor_countFROM ume.admin.org_units AS ume_unitTRAVERSE GRAPH FROM ume_unit EDGE_TYPE =
Employs TARGET_GRID = 'kogi://' AS workerGROUP BY ume_unit.org_unit_id;-- 6. Natural language
query (delegates to NL-to-HyperQL bridge)-- POST /api/v1/query/nl{ 'question': 'Which of my
active projects are over budget?',  'cube_context': ['kogi.portfolio.components'],
'return_query': true }
```

Query routing

Query type	PostgreSQL	ClickHouse	Neo4j	Notes
Single HyperRow read	Always	—	—	< 10ms p99 on warm cache
DimSlice, < 1M rows	Always	—	—	GIN/BTree index; < 200ms p99
DimFold, > 1M rows	—	Routed	—	Activated lazily at first large query
Graph BFS, < 1M nodes	Recursive CTE	—	—	Adequate for all domain-level deployments
Graph analytics (Louvain, PageRank)	—	—	GDS (KLNK only)	Platform-scale only
AS_OF, hot window	Hot EventLog buffer	—	—	< 500ms p99 for < 10K events

9. The AI Computation Model

Every domain system defines AI signals — health scores, risk scores, anomaly flags, predictions — as Tier-2 computed attributes on Hypercubes. These are written exclusively by registered AI engine plugins via WritebackService, carrying model ID, confidence score, and staleness marker. No human actor may write to a System-permission attribute.

Two computation tiers

Tier	Execution	When used	Latency	Example
Tier-1 (formula)	Synchronous evaluation at read time. Evaluated inline against the current HyperRow. Result never stored — always fresh.	Deterministic derivations from other fields	< 1ms	budget_remaining = budget_allocated − budget_spent; GRAPH_SUM('Contains', 'Outbound', 'budget')
Tier-2 (AI signal)	Asynchronous. Triggered by on_cell_mutation(). Computed in-process (simple models) or via remote engine (LLM). Written back via WritebackService.	ML models, LLM inference, anomaly detection	100ms–30s	health_score, risk_score, narrative_summary, compliance_prediction

The simplified AI pipeline

The previous specification required Kafka for all AI compute routing, a WritebackService gRPC endpoint, and separate processes for each AI engine (Scala 3 for kogi-engine, Go+Python for ume-engine). This is right at Enterprise scale but is the wrong default. The correct default is in-process.

	Previous spec (always)	This spec (tiered)
Simple signals (health score, risk score, anomaly)	Cell mutation → Kafka → separate engine process → gRPC WritebackService	Cell mutation → in-process plugin queue → HypercubePlugin.compute() in background thread pool → internal writeback
Heavy inference (LLM, NL-to-HyperQL)	Same Kafka pipeline	Kafka → remote engine process → gRPC WritebackService (same as before — only for heavyweight models)
Infrastructure requirement	Kafka + WritebackService gRPC + separate engine process always	Nothing extra for in-process; Kafka + gRPC only when a remote engine is registered

```
// In-process AI plugin — no Kafka, no gRPC, no separate service// This is correct for simple signals at Collaborative tierimpl HypercubePlugin for HealthScoreEngine {    fn on_cell_mutation(&self, cube_id: CubeId, coord: &DimCoordinate)        -> Vec<AttributeKey> {        // Which mutations trigger a recompute?        match coord.d2_str().as_deref() {            Some("status"|"budget_spent"|"risks") => vec!["health_score".into()],            _ => vec![],        }    }    fn compute(&self, row: &HyperRow, ctx: &ComputeContext) -> ComputeResult {        // ctx.neighbors gives in-memory graph access        let dep_count = ctx.neighbors.count(EdgeType::Dependency, Inbound);        let score = compute_health(row, dep_count);  // pure function        Ok(AiSignal { value: Number(score), confidence: 0.87, ... })    }}// Grid bootstrap — registered in-process, zero extra infragrid.plugin_registry.register(Arc::new(HealthScoreEngine::new()))?;// Remote AI engine (LLM-based) — only when genuinely needed// Activated by registering a RemoteAiEnginePlugin that talks gRPC// This brings Kafka and WritebackService gRPC into the stack
```

10. Technical Concerns

Six meaningful technical concerns are graded and addressed. The first two are resolved by this specification. The remaining four are real constraints that require conscious management.

Concern 1 — Infrastructure overhead (resolved)

The original specification only described the Enterprise deployment topology — PostgreSQL, Redis, Kafka, ClickHouse, Meilisearch, S3, plus five custom services. Most workloads do not need all of this simultaneously. This specification replaces that with a tiered model (Section 11) where infrastructure is activated only as scale demands.

> **Status:** Resolved by tiered deployment and lazy service activation in this specification.

Concern 2 — LWW on lifecycle states (resolved)

LWW status means a network partition could cause two nodes to hold different lifecycle states for the same entity, resolved by recency rather than governance. For regulated use cases this is a correctness defect. Resolved by shipping the Lattice CRDT in v1 (Section 5).

> **Status:** Resolved — Lattice CRDT for lifecycle states is v1, not v2.

Concern 3 — 255-node VectorClock ceiling (resolved)

The VectorClock = HashMap<NodeId, u64> ceiling of 255 nodes is removed by the Hybrid Logical Clock (Section 5). The HLC is 16 bytes per cell, uses a 32-bit node_id with no ceiling, and simplifies federation delta sync to a time-range scan.

> **Status:** Resolved — HLC replaces VectorClock in this specification.

Concern 4 — Serialization overhead per field

Writing a 30-field entity produces 30 HyperCell writes if not batched. Each cell carries coordinate, attribute map, HLC timestamp, actor ID, and EventLog metadata. This is richer than a simple column update. The mitigation is consistent use of write_cell_batch() in all bridge-layer write() implementations — one PostgreSQL transaction, one EventLog batch entry, one CrdtLog push.

> **Mitigation:** Always use batch writes in the bridge layer. Never write fields individually in a loop. Monitor p99 write latency; target < 50ms for a 30-field entity on a standard node.

Concern 5 — HyperQL expertise required

Cross-system queries, DimFold aggregations, and graph traversal require HyperQL fluency. The NL-to-HyperQL bridge helps end users but does not replace the need for platform engineers to understand N-dimensional query planning. The learning curve is real and proportional to dimensionality.

> **Mitigation:** Start all cubes at N=2. Build a library of standard HyperQL query templates for each domain. Ship the NL-to-HyperQL bridge early so end users can express intent in natural language. Only introduce N=3+ axes when the cross-sectional query pattern is understood and required.

Concern 6 — The spreadsheet metaphor has limits

Kogi and Ume have data that genuinely wants to be a spreadsheet substrate — entities with typed fields, time-series, ownership, computed aggregates. Qala's CI/CD pipelines, hermetic build environments, and SDE

provisioning workflows are infrastructure automation problems where GitOps tools already provide faster reconciliation. The $D_1 \times D_2$ entity model adds indirection for deeply nested or process-oriented data.

> **Mitigation:** For Qala, use Hypergrid for governance, audit, solution metadata, and CCR workflow — where the substrate genuinely helps. Integrate with existing GitOps tooling (ArgoCD, Flux) for SDE provisioning rather than replacing them. The SDE N=3 cube for version history is a good fit; continuous reconciliation of build environments is not.

11. Deployment Tiers

The infrastructure stack is tiered. Each tier is a subset of the next. The Grid runtime selects the backend stack from a GridProfile enum. Upgrading from Embedded to Collaborative requires a config change in Grid::new() and a data migration — the DomainStore codec and domain logic are unchanged. This is the single most important practical principle in this document.

Tier 1 — Embedded

Context	Solo · Mobile · Desktop · Offline-first · Developer tooling
Infrastructure	SQLite only — zero external services
Grid profile	GridProfile::Embedded
What works	Full Grid runtime, CRDT engine, HyperQL, Hypergraph, in-process AI plugins, Tier-1 formulas, EventLog
What does not work	Multi-node federation, CrossGridLink consent protocol, remote AI engines (LLM inference), collaborative editing WebSocket
Sync model	Delta sync via file export on reconnect. On next connection to a Collaborative node, all accumulated CrdtLog operations sync via standard CRDT merge.
Examples	Kogi mobile app, Kogi desktop, solo developer Qala, CLI tooling, local test environment

Tier 2 — Collaborative

Context	Team · Small org · Cooperative · Up to ~50 concurrent nodes
Infrastructure	PostgreSQL + Redis — required. Kafka, ClickHouse, Meilisearch — optional, lazy-activated
Grid profile	GridProfile::Collaborative
What works	Everything in Embedded, plus: multi-user concurrent editing (WebSocket), CrossGridLink within the same Grid, simple in-process AI signals, namespace resolution, Spaces, full HyperQL including DimFold (routes to ClickHouse when needed)
Lazy activation	Kafka: activated on first federation peer handshake. ClickHouse: activated when a DimFold query estimate exceeds 1 million rows. Meilisearch: activated on first searchable cube registration.
AI at this tier	In-process HypercubePlugin.compute() in background thread pool. Remote AI engines (Scala 3 kogi-engine, gRPC WritebackService) are available but not required.
Examples	Kogi cooperative (20–200 members), Ume small-org deployment, Qala factory team

Tier 3 — Enterprise

Context	Large org · Platform scale · Full federation · 50+ concurrent nodes
Infrastructure	Full stack: PostgreSQL + Redis + Kafka + ClickHouse + Meilisearch + S3 + WritebackService gRPC + FederationCoordinator service + separate AI engine processes
Grid profile	GridProfile::Enterprise

What works	Everything in Collaborative, plus: full CRDT federation across many nodes, CrossGridLink between separate Grids (Kogi↔Ume↔Qala), remote heavyweight AI engines (LLM inference, kogi-engine in Scala 3), EventLog cold archive to S3, ClickHouse for large-scale analytics, Neo4j for KLNK at 1M+ nodes
Clock	HLC replaces VectorClock — 255-node ceiling eliminated
Examples	Production Kogi platform, enterprise Ume org, regulated-industry Qala with pharmaceutical Domain Pack

Service activation sequence

Service	Activation trigger	Without it
PostgreSQL	Collaborative tier — always	Grid runs on SQLite (Embedded) or fails to start (Collaborative+)
Redis	Collaborative tier — always	Grid runs without cell cache; all reads go to PostgreSQL; CrdtLog writes to PostgreSQL directly (slower)
Kafka	First federation peer registered OR first remote AI engine registered	Grid operates standalone; in-process AI only; no multi-Grid sync
ClickHouse	First DimFold/DimExpand query that estimates > 1M cell scan	Query routes to PostgreSQL; may be slow for very large aggregations
Meilisearch	First HypercubePlugin registers a cube as searchable	Full-text search returns empty results; namespace resolution still works via PG
S3	EventLog hot partition reaches 500K entries OR first snapshot request	EventLog stays in PostgreSQL; cold AS_OF queries may be slower at high volume
WritebackService gRPC	First remote AI engine plugin registered	In-process AI engines still work; Kafka-based engines are unavailable
FederationCoordinator	Second Grid registered as federation peer	Single-Grid deployments do not need it; CrossGridLink within one Grid works without it
Neo4j	Graph node count exceeds 1M in any Space subgraph	BFS/DFS via PostgreSQL recursive CTE — correct, adequate below 1M nodes

12. The DomainStore Contract

Every domain system must implement the DomainStore codec. This is the bridge layer — the only place domain types touch Hypergrid primitives. Four methods. Nothing else.

```
trait DomainStore<Entity, EntityId> {    // 1. Register all Hypercubes and attribute schemas — idempotent, runs at startup    fn bootstrap(grid: &mut Grid) -> HypergridResult<()>;    // 2. Serialize domain entity → HyperCells and batch write to grid    fn write(grid: &mut Grid, cube_id: CubeId, entity: &Entity, actor: &str)        -> DomainResult<()>;    // 3. Read HyperCells → deserialize into domain entity (None if not found)    fn read(grid: &Grid, cube_id: CubeId, id: EntityId)        -> DomainResult<Option<Entity>>;    // 4. Check entity existence without full deserialization    fn exists(grid: &Grid, cube_id: CubeId, id: EntityId) -> bool;}// The write() implementation must://   - Use write_cell_batch() — never individual write_cell() in a loop//   - Include all mandatory system fields on every write//   - Use TransitionState (not SetAttr) for Lattice fields//   - Use AddToSet (not SetAttr) for OR-Set fields
```

Mandatory attribute keys

Every primary Hypercube in every domain system must register the following attribute keys with the specified CRDT semantics and permission tiers. These are the platform contract — without them, federation, unified audit, and governance cannot function.

Attribute key	Type	CRDT	PermissionTier	Purpose
created_at	DateTime	LWW	System	Entity creation timestamp — written once, never updated
updated_at	DateTime	LWW	System	Last mutation timestamp — updated on every write by orchestrator
last_actor	Text	LWW	System	NodeId of last writer — HLC tiebreak context for debugging
hlc_timestamp	BigInt (HLC)	LWW	System	Causal timestamp snapshot at last write — required for federation merge
version	Text	MaxRegister	Editor	Semantic version string — always takes highest observed value
status	Json	Lattice	Editor	Lifecycle status enum — defined by each domain's LatticeOrder
visibility	Json	LWW	Owner	Private | Protected | Public
owners	Json	OR-Set	Owner	Vec<UserId> — primary ownership; concurrent adds both survive
tags	Json	OR-Set	Contributor	Freeform string tags — concurrent adds both survive
policy_ids	Json	LWW	Admin	Attached governance policy IDs
namespace_path	Text	LWW	System	NamespacePath URI — required for cross-system linking

13. Performance Targets

Reference targets on a standard production node (8 vCPU, 32 GB RAM, NVMe SSD, PostgreSQL 16). All cache hit targets assume a warm Redis L1 cache. All cache miss targets assume a cold cache with a direct PostgreSQL read.

Operation	p50	p99	Notes
Single HyperCell read (Redis hit)	< 1ms	< 5ms	Redis HGET on cell key
Single HyperCell read (PG miss)	< 5ms	< 20ms	PostgreSQL primary key lookup
HyperRow read, 30 fields (Redis hit)	< 1ms	< 5ms	Redis HGETALL on row key
HyperRow read, 30 fields (PG miss)	< 8ms	< 40ms	PG D_1 scan; batch SELECT
Batch write, 30 fields	< 12ms	< 50ms	One PG transaction; one EventLog batch
DimSlice query, < 1K rows	< 35ms	< 200ms	PG GIN/BTree index
DimFold, N=3, < 100K rows	< 500ms	< 2s	PG window functions
DimFold, N=3, > 1M rows	< 1s	< 6s	ClickHouse (routed lazily)
Graph BFS, depth=3, fan-out=10	< 45ms	< 200ms	PG recursive CTE; ~1K nodes explored
AS_OF, hot window (< 10K events)	< 80ms	< 500ms	In-memory EventLog replay
AS_OF, cold (PostgreSQL)	< 300ms	< 1.5s	PG EventLog partition scan
CrossGridLink delta sync (real-time)	< 300ms	< 1s	Kafka + ShadowCell PG write + WS push
In-process AI (simple signal)	< 80ms	< 350ms	HypercubePlugin.compute() in thread pool
Remote AI (LLM inference)	< 800ms	< 3s	Kafka + gRPC WritebackService + LLM call
NL-to-HyperQL translation	< 200ms	< 800ms	LLM inference; 4K context window

Scale limits

Dimension	Limit	Mitigation when limit approached
Max entities per Hypercube	~1 billion (PG HASH partition per Grid shard)	Shard by grid_id
Max attribute keys per Hypercube	1,024 (AttributeKeyRegistry hard cap)	Split cube into primary + extension cube; join via D_1 key
Max dimensions per Hypercube	16 (HLC-compatible); > 6 not recommended for user-facing views	Restructure: use separate cubes joined via graph edges
Max federation peers (HLC)	No ceiling — 32-bit node_id (4 billion)	Previous VectorClock limit of 255 is eliminated by HLC
Max ClickHouse throughput	~100M cells/second for DimFold	Horizontal ClickHouse scaling
Graph PG CTE performance	Degrades past ~1M nodes explored per traversal	Route to Neo4j for KLNK at > 1M nodes

14. Security Pipeline

Every mutation flows through a multi-layer security pipeline before persistence. The layers are sequential — a write cannot skip any of them.

Layer	Mechanism	What it enforces
1. API gateway auth	JWT (RS256) or API key; mTLS for service-to-service gRPC	Identity established before any resource access
2. PermissionTier enforcement	caller.tier ≥ attr.write_permission for every attribute key	Editors cannot write Manager fields; humans cannot write System fields
3. VisibilityMask	Applied on every read; identity and tenancy partition isolation	TenantPartition isolation; SplitPolicy for cross-partition access
4. PolicyEngine	GovernanceEngine evaluates all registered PolicyEngine plugins per write	Domain-specific constraints: budget caps, approval gates, regulatory rules
5. CRDT validation	LWW: reject future HLC timestamps; Lattice: reject invalid transitions; OR-Set: validate tag uniqueness	Data invariants enforced before persistence
6. EventLog (audit trail)	Every write produces an immutable EventLog entry with full before/after values	PostgreSQL RLS + application code both prevent UPDATE/DELETE on event_log
7. Field-level encryption	AES-256-GCM via KMS (AWS KMS or HashiCorp Vault) for PII fields	Owner-level encryption: only owner can decrypt private fields; no raw keys in application code

15. Bootstrap Sequence

When Grid::new(config) is called, the initialization sequence runs in seven phases. Phases 1–5 are blocking — the Grid refuses traffic until they complete. Phases 6–7 are async — the Grid is Ready before they finish.

Phase	Operation	Blocking?	Failure handling
1	Config validation and GridProfile selection — Embedded / Collaborative / Enterprise	Yes	Fatal: refuse to start; log all missing config keys
2	Storage connectivity — open PG pool; test Redis PING (if Collaborative+); apply DB migrations; verify Kafka if configured	Yes	Retry with exponential backoff; fatal after 5 minutes
3	Plugin registry init — load all HypercubePlugin implementations; call plugin.on_load(); build capability index	Yes	Degraded plugins: log warning, continue. Missing Required plugins: fatal.
4	Grid bootstrap — call DomainStore::bootstrap() for all domain systems; register Hypercubes and attr schemas; CREATE TABLE IF NOT EXISTS	Yes	Idempotent: re-registering existing cubes/attrs is a no-op
5	EventLog recovery — query CrdtLog for unapplied operations; re-apply via CRDT merge engine; advance HLC to recovered state	Yes	CRDT merge on conflict; log RecoveryCompleted event
6	Namespace warm-up, federation handshake, AI engine health-check	No (async)	Standalone mode if peers unreachable; cache starts cold; AI degrades gracefully
7	Ready — readiness probe returns HTTP 200; API gateway routes traffic	—	Only reached if phases 1–5 succeeded

16. Summary and Design Decisions

This section collects every deliberate design decision in this specification in one place, with the rationale for each.

Decision	Choice	Rationale
Causal clock	Hybrid Logical Clock (HLC) — 16 bytes, no node ceiling	VectorClock's 255-node ceiling and O(N) per-cell storage are the wrong constraints for a platform expected to reach tens of thousands of federation nodes. HLC is proven in CockroachDB, MongoDB, and TiKV with identical causal correctness properties.
Lifecycle state CRDT	Lattice CRDT — ship in v1	LWW on lifecycle state is a correctness defect under network partition for any governance-meaningful field. The Lattice implementation is fully specified. Deferring it to v2 adds no simplification and creates regulatory risk for Qala use cases.
AI pipeline default	In-process HypercubePlugin in background thread pool	Simple signals (health score, risk score, anomaly flag) compute in milliseconds and do not need a separate process, Kafka routing, or gRPC. Remote engines are reserved for heavyweight LLM inference and activated lazily.
Storage tier default	SQLite (Embedded) / PostgreSQL+Redis (Collaborative) / full stack (Enterprise)	The previous spec only described the Enterprise topology. Solo users and small cooperatives should not need to stand up six services before seeing any value. The backend is pluggable via the CellStoreBackend trait.
Lazy service activation	Kafka, ClickHouse, Meilisearch activate on first demand	Each service has a clear activation trigger. Before that trigger, the Grid runs without it. This cuts Collaborative tier infrastructure from six services to two.
Graph backend	PostgreSQL recursive CTE below 1M nodes; Neo4j above 1M (KLNK only)	Adding Neo4j to every Grid deployment is unnecessary for domain-scale graphs. The PG CTE is adequate for all individual domain deployments. Neo4j enters only when the KLNK economic graph reaches platform scale.
Batch writes in bridge layer	Always — write_cell_batch() is mandatory, not optional	A 30-field entity written individually produces 30 PG round-trips, 30 EventLog inserts, and 30 Redis writes. Batch collapses this to one of each. This is too important to leave as an optimization hint.
Dimension axis discipline	N=2 default; D_3+ only when cross-sectional queries are known and required	Speculative axis additions add storage overhead and query planning complexity without adding value. The discipline is: a new axis is justified by a specific query pattern, not a theoretical data model.
CrossGridLink consent model	Six-phase protocol with target-owner control over mirrored attrs	Data sovereignty is the design principle. The requesting Grid sees only what the target owner explicitly consents to share. Revocation is always available. This is non-negotiable for the KLNK economic graph.
Qala and GitOps	Integrate with ArgoCD/Flux for SDE provisioning; use Hypergrid for governance, audit, and CCR workflow only	Continuous reconciliation is not a problem Hypergrid is designed to solve faster than existing GitOps tools. The correct scope for Qala on Hypergrid is governance tracking, not runtime infra automation.

End of Document — Hypergrid Design and Architecture v1.0

Hypergrid NDSS · Apapo Platform · Kogi · Ume · Qala · v1.0 · March 2026 · Confidential — Internal Use Only